

Часть 4

NP-трудность и NP-полнота. Существование NP-полного языка. Теорема Кука-Левина

Опр Язык B является NP-трудным, если $\forall A \in NP \cup A \leq_p B$

Опр Язык B является NP-полным, если $B \in NP$ и B - NP-трудный.

Утв $B \in P$; B - NP-трудный $\Rightarrow P = NP$

Утв A - NP-трудный; $A \leq_p B \Rightarrow B$ - NP-трудный.

Зам Множество NP-полных языков будем обозначать NPC .

Теорема $\exists L_{NP} = \{ \langle N, x, 1^t \rangle \mid \text{НМТ } N \text{ принимает } x \text{ за время } t \}$. Тогда

$L_{NP} \in NPC$

$\square$ Выберем произвольный $L \in NP$. Он распознаётся какой-то НМТ N_L за время

$p(|x|)$, где $p(|x|) = O(\text{poly}(|x|))$. Тогда, выписывая за полином функцию f делаем следующее: $f_{L(x)} = (N_L; x, 1^{p(|x|)})$. Из построения следует, что

$x \in L \Leftrightarrow f(x) \in L_{NP}$. Таким образом, $\forall L \in NP \cup L \leq_p L_{NP} \Rightarrow L_{NP}$ - NP-трудный.

$L_{NP} \in NP$, т.к. можно смоделировать работу N на x в течение t шагов, и это будет полиномиально относительно длины входа. $\Rightarrow L_{NP} \in NPC$ ■

Теорема (Кука-Левина)

$SAT = \{ \varphi \mid \varphi - \text{выполнимая булева формула} \}$. $SAT \in NPC$.

$\square$ 1) Сначала докажем, что $SAT \in NP$. Для этого надо придумать верификатор и сертификат. В качестве сертификата будет набор из значений для каждой переменной.

вына

ной. Т.к. каждая ~~переменная~~ переменная встречается в формуле хотя бы один раз, то количество переменных $n \leq |\varphi|$. $\Rightarrow$ длина сертификата полиномиальна относительно

длины входа. Верификатор же будет ~~будет~~ брать этот фиксированный набор значений, подставлять в формулу и вычислять. Это тоже работает за полином от $|\varphi| \Rightarrow$

$\Rightarrow SAT \in NP$

2) Теперь докажем, что SAT - NP-трудный.

Выберем произвольный $L \in NP$. Хотим доказать, что $L \leq_p SAT$. То есть

построить f - полиномиально вычисляемую от $|x|$ такую, что $x \in L \Leftrightarrow f(x) \in SAT$

Т.к. $L \in NP$, то $\exists$ верификатор V . Тогда p -сводимость будет заключаться в

том, что мы по слову x , зная верификатор V , хотим построить булеву формулу

φ такую, что φ -выполнима $\Leftrightarrow \exists$ сертификат s для верификатора V . Также

для языка L есть некий полином $p(|x|)$, который ограничивает сверху время работы верификатора.

Давайте сделаем верификатор специального вида (такой всегда $\exists$):

1) Верификатор - однокленточная ДМТ с лентой, бесконечной в одну сторону.

2) Алфавит верификатора $A = \{ \blacktriangleright; \#; 0; 1; \blacktriangleleft; \wedge \}$.

$$p(|x|) - q(|x|) - |x| - 4$$

3) Входная лента всегда выглядит так:

$\blacktriangleright \underbrace{010\dots 0}_{\substack{|x| \\ \text{бинарный код } x}} \# \underbrace{110\dots 0}_{\substack{q(|x|) \\ \text{сертификат } s}} \# \underbrace{11\dots 1}_{\substack{p(|x|) - q(|x|) - |x| - 4}} \blacktriangleleft$

Затем мы так сделали? Лента, бесконечная в одну сторону, гарантирует, что мы

не выйдем левее символа $\blacktriangleright$. Также в качестве входа мы подали верификатору

x и s . $|s| = q(|x|)$, то есть длина сертификата фиксированна. Такое
 отражение можно дополнительно наложить, чтобы было верное определение NP
 (см. задачу 7 из 8/3). Далее мы заполним ленту символами так, чтобы
 всего на ленте было $p(|x|)$ символов. Т.к. верификатор работает $\leq p(|x|)$ шагов,
 то он никогда не сможет выйти правее символа $\blacktriangleleft$. Таким образом $\nexists$
 конфигурация верификатора V может быть представлена как набор символов
 только на указанных ячейках (т.е. $p(|x|)$ ячеек) и перед текущим символом, есть
 один символ состояния. $\Rightarrow$ длина корректной конфигурации $V \leq p(|x|) + 1$,
 далее справа на ленте всегда только $\perp$.

Теперь возьмём все символы из $A \cup Q$ - алфавит и множество состояний V
 соответственно. Пусть $|A \cup Q| = N$ - константа, не зависящая от x . Тогда для
 кодировки всех символов из $A \cup Q$ хватит $m = \lceil \log_2 N \rceil$ бит, то есть вместо
 каждого символа мы будем писать его двоичный код длины m .

Построим таблицу размером $p(|x|) \times m(p(|x|) + 1)$, т.е.

$x_{11} x_{12} \dots x_{1m}$	$x_{1m+1} x_{1m+2} \dots x_{12m}$	$\dots$	$x_{1m(p(x)+1)} \dots x_{1m(p(x)+1)m}$
$x_{21} x_{22} \dots x_{2m}$			
$\vdots$			
$\vdots$			
$\vdots$			
$x_{p(x)+1} \dots x_{p(x)+1m}$			

Пусть каждая ячейка - уникальная переменная из булевой формулы $\Rightarrow$ будет $p(x_i) = m(p(x_i)+1)$ переменных. Хотим как-то построить формулу, но она будет выполнима $\Leftrightarrow$ каждая строка таблицы - корректная конфигурация V , все $p(x_i)$ строк образуют корректный протокол V и V принимает x .

Назвём создавать эту булеву формулу. Сначала проверим корректность первой строки. Для этого мы смо провераем, что первые m символ^{ов} - корректное двоичное кодирование символа $\blacktriangleright$, потом аналогично проверяем на корректность все символы $\chi; \#; \wedge$ и $\blacktriangleleft$ и из первой строки мы пропускаем только сертификат, ведь мы это не знаем. Для понимания, как это будет выглядеть в формуле. Пусть $m=5$;

$\blacktriangleright = 10011$. Тогда проверка, что первые 5 символов задают $\blacktriangleright$ будет выглядеть, как $(x_{11} \equiv 1) \wedge (x_{12} \equiv 0) \wedge (x_{13} \equiv 0) \wedge (x_{14} \equiv 1) \wedge (x_{15} \equiv 1)$. Проверка остальных символов делается

аналогично. Все разделители между проверками всех ячеек - конъюнкции. Так мы проверили корректность первой конфигурации. Научимся проверять корректность всех остальных.

Рассмотрим ζ подряд идущих ^{блока из m клеток} ~~ячеек~~ в строке и один ^{блок} ~~символ~~ ниже, как показано далее:



Утверждается, что по верхним ζ ^{блокам} ~~символам~~ можно гарантированно

проверить этот ^{блок} ~~символ~~ снизу. Рассмотрим все ситуации. Далее ~~а, в, с, d, q~~ a, b, c, d, q

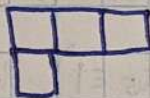
$a, b, c, d, a', b', c', d' \in A; q, q' \in Q$. За y обозначим значение в блоке снизу.

Тогда: $abcd \Rightarrow y \equiv b$; $qbcd \Rightarrow \begin{cases} y \equiv b'; \text{ если } \delta(q, b) = (q', -1; b') \\ y \equiv q'; \text{ если } \delta(q, b) = (q', 1; b') \end{cases}$

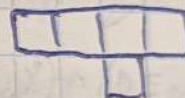
$$\begin{aligned}
 aqc \Rightarrow & \begin{cases} y \equiv a, & \text{если } \delta(q, c) = (q', -1; c') \\ y \equiv q', & \text{если } \delta(q, c) = (q', 0; c') \\ y \equiv c', & \text{если } \delta(q, c) = (q', 1; c') \end{cases} ; \quad acqb \Rightarrow \begin{cases} y \equiv q', & \text{если } \delta(q, a) = (q', -1; b') \\ y \equiv c, & \text{если } \delta(q, a) = (q', 0; b') \end{cases}
 \end{aligned}$$

$$absc \Rightarrow y \equiv b$$

Все такие проверки делаются побитовым сравнением, которое если оформляется через импликацию, то условие $\rightarrow$ проверка при этом условии. Для каждого блока надо проверить N^4 значений верхних блоков и далее расписать проверки, то есть таблица формулы для каждого блока будет длины $O(1)$, так не зависит от n , а зависит только от V . Так, мы можем проверить корректность всех символов, кроме самого левого в строке и двух самых правых. Для них можно провести аналогичные рассуждения, рассмотрев:



— для левого;



— для

края самого правого;  — для правого.

Осталось проверить, что V принял слово. Для этого достаточно проверить, что один из блоков в нижней строке — код q_{yes} — принимающего блока. То есть проверим, что хотя бы один из блоков — q_{yes} — побитовое сравнение с конъюнкцией для проверки символов внутри блока и дизъюнкцией между проверками блоков.

Итак, за $O(p(n) \cdot t(p(n) + 1)) = O(p(n) \cdot t(n))$ построим формулу φ такую, что

$$\varphi \text{ — выполнима} \Leftrightarrow x \in L \Rightarrow L \leq_p SAT \quad \forall L \in NP \Rightarrow SAT \text{ — NP-трудный} \Rightarrow$$

$$\Rightarrow SAT \in NPC \blacksquare$$

Теорема

SAT

Тогда и

Теорема (8/9) Пусть $3SAT = \{\varphi \mid \varphi - \text{выполнимая булева формула в виде 3КНФ}\}$.

$$SAT \leq_p 3SAT$$

Тогда из этой теоремы следует, что $3SAT \in NPC$